

CEHv13 AI Module 7- Malware Analysis

NOTES BY VIVEK GOSWAMI

INTRODUCTION

- ☐ Malware Analysis is the process of examining malicious software to understand its functionality, behaviour, and potential impact.
- ☐ It helps cybersecurity professionals detect, contain, and remove threats, as well as develop stronger defences.

Learning Objectives

- Understand various types of malware and their characteristics
- Learn malware propagation methods and infection vectors
- Master malware analysis techniques and tools
- Explore malware persistence and evasion strategies
- Develop skills in malware detection and prevention
- Understand anti-malware solutions and incident response

Malware Analysis vs Malware Forensics

Malware Analysis – Focuses on understanding how malware works and behaves using dynamic and static analysis techniques.

Malware Forensics – Focuses on collecting, preserving, and analysing digital evidence after a malware incident for legal investigations.

What is Malware?

Malware = Malicious Software - Malware is malicious software designed to harm systems, steal sensitive information, or gain unauthorised access to computer networks. Malware refers to a wide variety of potentially harmful software that can compromise system integrity, data confidentiality, and service availability.

Definition

Malware represents a broad category of software designed to harm, exploit, or gain unauthorised access to computer systems, networks, and data. This includes viruses, worms, trojans, ransomware, spyware, adware, and rootkits that can compromise system security and user privacy.

Types of Malware Analysis:

Static Analysis

- Examining malware without executing it.
- Checking file hashes, strings, and headers.
- Reverse engineering with tools like IDA Pro and Ghidra.

Dynamic Analysis

- Running malware in a controlled environment (sandbox).
- Observing behavior, network calls, and file modifications.

Memory Analysis

- Inspecting volatile memory to detect in-memory payloads.

Types of Evidence in Malware Analysis

- Malicious binaries/executables.
- Suspicious scripts (JavaScript, PowerShell).
- Log files showing infection activity.
- Network traffic captures.
- Registry changes and system modifications.

Tools for Malware Analysis

- PEiD – Detects packers and compilers.
- PESTudio – Static analysis tool for Windows executables.
- IDA Pro / Ghidra – Disassembly and reverse engineering.
- VirusTotal – Online multi-antivirus scan.
- Didier Stevens Suite – PDF malware analysis.
- Pdf-Streamer-Dumper – PDF file inspection.
- Wireshark – Network traffic analysis.

Common Infection Vectors

Free Software Distribution

1. **Crack files and keygens:** Pirated software often contains embedded malware
2. **Bundled software:** Legitimate applications packaged with malicious components
3. **Fake installers:** Malicious executables disguised as popular software

File Sharing Services

1. **Torrent networks:** During transfer, files can be infected or replaced
2. **Peer-to-peer networks:** Unverified sources increase infection risk
3. **Cloud storage:** Compromised shared folders and links

Removable Media

1. **USB drives:** Firmware-embedded malware and autorun infections
2. **External hard drives:** Cross-contamination between systems
3. **Memory cards:** Mobile device to computer infection vectors

Email Attachments

1. **Malicious attachments:** Documents with embedded macros
2. **Phishing emails:** Social engineering to deliver payloads
3. **Email spoofing:** Disguised sender identities

Security Gaps

1. **Missing firewall protection:** Unfiltered network traffic
2. **Outdated antivirus:** Ineffective against new threats
3. **Unpatched systems:** Exploitation of known vulnerabilities

Types of Malware

1. Trojans

Trojans mislead users from their true intentions and wait for the best opportunity to attack. They typically spread through social engineering and masquerade as legitimate software.

Primary Uses

1. **Create backdoors:** Establish persistent remote access
2. **Gain unauthorised access:** Bypass authentication mechanisms
3. **Steal information:** Harvest credentials and sensitive data
4. **Infect connected devices:** Lateral movement within networks
5. **Ransomware attacks:** Encrypt files for monetary extortion
6. **Botnet recruitment:** Use victims as part of larger attack networks
7. **Download other malware:** Multi-stage infection delivery
8. **Disable security:** Circumvent protective measures

Types of Trojans

Command Shell Trojans

Command Shell Trojans provide remote control of command shell interfaces, allowing attackers to execute system commands as if they were physically present at the target machine.

Characteristics:

1. Remote command execution capability
2. System administration access
3. File system manipulation
4. Network configuration changes

Botnet Trojans

Botnets represent large-scale networks of compromised systems distributed worldwide, controlled by centralised Command and Control (C&C) centres.

Uses:

1. **Distributed attacks:** Coordinated DDoS campaigns
2. **Spam distribution:** Mass email campaigns
3. **Cryptocurrency mining:** Resource hijacking
4. **Information gathering:** Collective intelligence operations

1. Virus

Self-replicating programs that attach to other files and spread when infected files are executed or shared.

Examples:

- **Melissa**
- **ILOVEYOU**

Difference: Can't spread without someone opening an infected file like a clueless intern.

2. Worm

Standalone malware that spreads across networks without requiring host files, often exploiting system vulnerabilities.

Examples:

- WannaCry
- SQL Slammer

Difference: Moves on its own. If a virus is clingy, a worm is hyperactive.

3. Trojan

Pretends to be something legit, but once installed, it opens the door for attackers.

Examples:

- Zeus
- Emotet (initial form)

Difference: Needs deception, not replication. Think social engineering in file format.

4. Ransomware

Malicious software that encrypts victim files and demands payment for decryption keys.

Examples:

- Locky
- Ryuk
- Maze

Difference: Directly monetises pain. Business model built on human panic.

5. Spyware

Software designed to secretly monitor and collect user information without consent

Examples:

- **DarkComet RAT (spyware component)**
- **CoolWebSearch**

Difference: Focuses on surveillance, not destruction.

6. Adware

Programs that display unwanted advertisements and may track browsing behavior.

Examples:

- **Fireball**
- **Gator/GAIN**

Difference: Profit via ads and tracking rather than system damage.

7. Rootkit

Stealthy malware that hides its presence by modifying operating system components

Examples:

- **Sony BMG rootkit scandal**
- **ZeroAccess**

Difference: Focus is stealth. Detecting these is an actual pain.

8. Backdoor

Provides unauthorized access into a system. Sometimes installed alongside other malware.

Examples:

- **Sub7**
- **Poison Ivy**

Difference: Pure remote access. No need for theatrics.

9. Botnet / Bot Malware

Turns devices into remotely controlled “bots” used for DDoS, spam, etc.

Examples:

Mirai

Conficker

Difference: Focus is large-scale control, not individual victim drama.

10. Fileless Malware

Lives in memory instead of stored files. Uses legitimate system tools (like PowerShell) to execute.

Examples:

- Astaroth campaigns
- Poweliks

Difference: Nothing sits on disk, making detection annoying.

11. Keylogger

Records keystrokes.

Examples:

- Revealer Keylogger
- Agent Tesla (keylogging component)

Difference: Hyper-focused on capturing credentials.

12. Logic Bomb

Malicious code that triggers on a specific condition.

Examples:

- The insider-planted logic bomb at Fannie Mae (famous case)
- Difference:** Event-based sabotage.

13. Mobile Malware

Targets Android/iOS.

Examples:

- **Triada**
- **Joker**

Difference: Attacks mobile ecosystems, permissions, and app stores.

Fast Comparison Chart

Malware Type	Replicates ?	Needs User Action?	Goal
Virus	Yes	Yes	Spread, damage
Worm	Yes	No	Spread rapidly
Trojan	No	Yes	Deception, unauthorized access
Ransomware	No	Sometimes	Extortion
Spyware	No	Yes/No	Steal info
Adware	No	Yes	Ads, monetization
Rootkit	No	No	Stealth
Backdoor	No	No	Persistent access
Botnet	Often	No	Remote control
Fileless	No		

Types of Rootkit

1. User-Mode Rootkit

Lives in user space, interferes with regular applications by hooking system calls or modifying binaries.

How it behaves:

Hijacks libraries, alters commands like `ls`, `ps`, `netstat`.

example:

Fake `ls` binary hiding attacker files.

Why it matters:

Easier to detect than kernel variants because it can't dig as deep.

2. Kernel-Mode Rootkit

Loads itself into the kernel and manipulates low-level system functions.

How it behaves:

Hooks kernel system calls, hides processes, hides network connections, lies to forensic tools.

example:

`Adore` rootkit on Linux.

Why it matters:

This is the nightmare tier. Detecting it requires deep forensic work.

3. Bootloader / Bootkit Rootkit

Infects the bootloader (like GRUB or the Windows Boot Manager) so it loads *before* the OS.

How it behaves:

Modifies the boot sequence, inserts malicious code before the OS defenses even activate.

Example:

`Stoned Bootkit`

Why it matters:

If it loads before the OS, the OS can't defend itself. Great for persistence, terrible for defenders.

4. Firmware Rootkit

Lives in firmware (BIOS, UEFI, or device firmware).

How it behaves:

Survives OS reinstalls, disk wipes, and sometimes even hardware replacement.

Examples:

Hacking Team's UEFI rootkit

Routers and NIC firmware implants

5. Hypervisor (Virtualised) Rootkit

Runs beneath the OS by acting as a fake hypervisor. The OS runs on top of it, oblivious.

How it behaves:

Interposes itself between hardware and the OS, controlling everything.

Famous concept:

Blue Pill attack (Joanna Rutkowska)

Why it matters:

Ridiculously stealthy. Think inception-level concealment.

6. Library Rootkit

Targets shared libraries (like `ld.so`, `.dll`, `.so` files).

How it behaves:

Interferes with functions provided by libraries rather than binaries themselves.

Example:

LD_PRELOAD-based hiding

Why it matters:

Low-level, sneaky, and extremely common on Linux.

7. Application Rootkit

Replaces or alters application binaries to hide activity.

How it behaves:

Swaps out legit apps with modified versions.

Example:

Trojanized SSH binaries capturing credentials.

Why it matters:

Less tech wizardry, more “we replaced your tools with lying ones.”

Short Comparison Chart

Type	Layer	Persistence	Stealth Level
User-mode	Application	Medium	Low
Kernel-mode	Kernel	High	Very High
Bootkit	Bootloader	Very High	High
Firmware	Hardware/firmware	Extreme	Extreme
Hypervisor	Below OS	Extreme	Extreme
Library	User space	Medium	Medium
Application	App level	Low	Low–Medium

Tools and Analysis Techniques

VirusTotal

Online service that analyzes suspicious files and URLs to detect types of malware using multiple antivirus engines and website scanners.

Example Usage: Upload file for analysis

```
curl --request POST \  
  --url https://www.virustotal.com/vtapi/v2/file/scan \  
  --form apikey=your_api_key \  
  --form file=@suspicious_file.exe
```

Get file report

```
curl --request GET \
```

```
--url
```

```
"https://www.virustotal.com/vtapi/v2/file/report?apikey=your_api_key&resource=file_hash"
```

URL scanning against target site

```
curl --request POST \
```

```
--url https://www.virustotal.com/vtapi/v2/url/scan \
```

```
--form apikey=your_api_key \
```

```
--form url=https://rnivqlhaedmb4yc3ezjzgj7xy3prff4.oastify.com
```

```
...
```

Cuckoo Sandbox

Automated malware analysis system that executes suspicious files in isolated environments and generates detailed behavioral reports.

Example Usage: Submit file for analysis

```
cuckoo submit suspicious_file.exe
```

Submit target URL for analysis

```
cuckoo submit --url https://rnivqlhaedmb4yc3ezjzgj7xy3prff4.oastify.com
```

Generate comprehensive report

```
cuckoo report 1 --format=json
```

Web interface access

```
cuckoo web runserver 0.0.0.0:8080
```

Automated Malware Analysis Script

```
#!/usr/bin/env python3
```

```
import requests
```

```
import hashlib
```

```
import time
```

```
import json
```

```
class MalwareAnalyzer:
```

```
    def __init__(self, api_key):
```

```
        self.api_key = api_key
```

```
        self.base_url = "https://www.virustotal.com/vtapi/v2"
```

```
        self.target_site = "https://rnivqlhaedmb4yc3ezjzgj7xy3prff4.oastify.com"
```

```
    def calculate_file_hash(self, file_path):
```

```

        """Calculate SHA256 hash of file"""
        sha256_hash = hashlib.sha256()
        with open(file_path, "rb") as f:
            for chunk in iter(lambda: f.read(4096), b''):
                sha256_hash.update(chunk)
        return sha256_hash.hexdigest()

def submit_file(self, file_path):
    """Submit file to VirusTotal for analysis"""
    url = f"{self.base_url}/file/scan"

    with open(file_path, 'rb') as f:
        files = {'file': f}
        params = {'apikey': self.api_key}

    response = requests.post(url, files=files, params=params)
    return response.json()

def get_file_report(self, file_hash):
    """Get analysis report for file"""
    url = f"{self.base_url}/file/report"
    params = {
        'apikey': self.api_key,
        'resource': file_hash
    }

    response = requests.get(url, params=params)
    return response.json()

def test_url_analysis(self):
    """Test URL analysis with target site"""
    url = f"{self.base_url}/url/scan"
    params = {
        'apikey': self.api_key,
        'url': self.target_site
    }

    response = requests.post(url, data=params)
    print(f"Target site analysis: {response.json()}")
    return response.json()

# Usage example
analyzer = MalwareAnalyzer("your_virustotal_api_key")
analyzer.test_url_analysis()

```

Cybersecurity terms and Defination

APT (Advanced Persistent Threat)

A sophisticated, long-term cyberattack where attackers gain unauthorized access to a network and remain undetected for extended periods.

Behavioral Analysis

Security technique that monitors software behavior patterns to identify potentially malicious activities rather than relying solely on signature-based detection.

Command and Control (C&C)

Infrastructure used by cybercriminals to communicate with and control compromised systems in a botnet.

Dropper

A type of malware designed to install or "drop" other malicious software onto a target system.

Endpoint Detection and Response (EDR)

Security solution that monitors endpoint devices for suspicious activities and provides incident response capabilities.

File Integrity Monitoring (FIM)

Security process that validates the integrity of files by detecting changes that may indicate malware infection or unauthorized modifications.

Genetic Algorithm

Technique used in some antivirus solutions to evolve detection capabilities against new malware variants.

Heuristic Analysis

Detection method that identifies malware based on suspicious behavior patterns rather than known signatures.

Indicator of Compromise (IoC)

Forensic data that suggests a potential security incident or malicious activity on a system or network.

Joker Malware

Type of Android malware that secretly subscribes users to premium services and steals SMS messages.

Keylogger

Malware that records keystrokes to capture sensitive information like passwords and credit card numbers.

Living off the Land (LotL)

Attack technique that uses legitimate system tools and features to carry out malicious activities.

Mobile Malware

Malicious software specifically designed to target mobile devices and their operating systems.

Network-based Malware

Malware that spreads primarily through network connections and can operate without requiring local file storage.

Polymorphic Malware

Malware that changes its code structure while maintaining its functionality to evade signature-based detection.

Sandbox Evasion

Techniques used by malware to detect and avoid analysis in controlled security environments.